

Introduction

The rise of the Games-as-a-Service (GaaS) business model and complex game worlds has led to a re-evaluation of the role of software quality assurance, placing research on automated code and feature synthesis into the spotlight [1]. Because game development is a highly diverse and multifaceted field, Coppola et al. proposed a taxonomy based on a systematic literature review. In their work, they identified 26 distinct test coverage metrics and categorized them into 6 main groups (user interface, gameplay, multimedia, operability & UX, performance, and reliability), thereby establishing a unified quantitative framework for evaluating and comparing automated game testing methodologies [2].

Throughout the evolution of game testing, two main paradigms have emerged: static analysis focusing on code coverage, and reinforcement learning (RL) simulating player behavior. Building upon the RL-based approach, research by Ferdous et al. investigates autonomous agent-based testing in complex 3D game environments. The authors bridge the challenges of sparse rewards and large state-action spaces using a curiosity-based Q-learning approach combined with high-level state abstraction. Their experiments in a 3D environment called Lab Recruits demonstrated that curiosity rewards used as intrinsic motivation—encouraging the exploration of new states over repeating previously visited ones—achieve significantly better functional entity and interaction coverage on more complex levels compared to traditional sparse-reward RL or random testing [3].

The CCPT (Curiosity-Conditioned Proximal Trajectories) algorithm combines curiosity-based exploration with imitation learning, enabling autonomous agents to explore complex 3D navigation environments in close proximity to expert demonstrations. Sestini et al. proposed a conditional intrinsic reward function that uses a weight parameter to regulate the balance between imitation and curiosity-driven exploration during testing. This approach not only results in more effective environmental coverage and higher bug discovery

rates compared to conventional RL-based methods, but also automatically identifies and highlights anomalous trajectories (e.g., those stemming from missing collision boxes or unusual physics behaviors) [4].

The emergence of Large Language Models (LLMs) has opened new horizons in software architecture testing. In research by Paduraru et al., the Code Llama model was intentionally fine-tuned on game development codebases, engine-specific APIs (such as Unity and Unreal Engine), and strongly typed languages (C# and C++). The authors applied a synthetic data generation method (execution feedback) and Retrieval Augmented Generation (RAG) context-building to bridge the lack of expert test databases. Their results confirmed that fine-tuned models (such as GameUnitGen) achieve significantly better compilation and execution accuracy, as well as higher branch and statement coverage than general-purpose base models with much larger parameter counts, while their lower resource requirements allow them to run efficiently in local development environments [5].

However, LLM-based testing agents often ignore the diverse strategies arising from individual human player personality traits, leading to repetitive testing patterns. To address this, Chen et al. introduced a framework called MIMIC, which integrates seven distinct player personality traits (e.g., cautiousness, aggressiveness, curiosity) into the agents’ decision-making processes. In their experiments, MIMIC achieved higher code and interaction coverage, better task completion rates, and significantly more diverse testing trajectories compared to existing state-space-based and LLM agents, more effectively uncovering rare edge-case bugs [6].

A critical challenge in game development is measuring and balancing a game’s objective difficulty curve. Xiao and Yang proposed a general framework that employs LLMs as autonomous agents to evaluate the relative difficulty of games. In their experiments, they found that although the absolute performance of LLM agents (e.g., win rate or number of attempts) may fall short of average human players, their guided decision-making shows a strong and statistically significant correlation with the relative difficulty experienced by human testers. Their findings highlight that LLMs can be used as proxy testers in early development stages to validate difficulty curves [7].

For the continuously updated GaaS model, Mu et al. introduced a framework named SMART (Structural Mapping for Augmented Reinforcement Testing). Using Large Language Models (LLMs), the method analyzes Abstract Syntax Trees (ASTs) and guides the RL agent towards gameplay objectives in a structure-aware manner, while adaptively exploring modified code

branches and boundary-value interactions. SMART achieves an outstanding branch coverage of up to 98% in modified codebases—far outperforming traditional RL methods’ 31–58% results—while maintaining a high task completion rate (79–98%) [8].

References

- [1] Cristiano Politowski, Fabio Petrillo, Yann-Gael Gueheneuc. 2021. A Survey of Video Game Testing
- [2] Riccardo Coppola, Tommaso Fulcini, Serenella Manzi, Francesco Strada. 2022. How to Measure Game Testing: a Survey of Coverage Metrics
- [3] Raihana Ferdous, Fitsum Kifetew, Davide Prandi, Angelo Susi. 2022. Towards Agent-Based Testing of 3D Games using Reinforcement Learning
- [4] Alessandro Sestini, Linus Gisslén, Joakim Bergdahl, Konrad Tollmar, Andrew David Bagdanov. 2024. Automated Gameplay Testing and Validation With Curiosity-Conditioned Proximal Trajectories
- [5] Ciprian Paduraru, Alin Stefanescu, Augustin Jianu. 2024. Unit Test Generation using Large Language Models for Unity Game Development
- [6] Yifei Chen, Sarra Habchi, Lili Wei. 2025. MIMIC: Integrating Diverse Personality Traits for Better Game Testing Using Large Language Model
- [7] Chang Xiao, Zixiaofan Yang. 2025. LLMs May Not Be Human-Level Players, But They Can Be Testers: Measuring Game Difficulty with LLM Agents
- [8] Enhong Mua, Minami Yoda, Yan Zhang, Mingyue Zhang, Yutaka Matsuno, Jialong Li. 2026. Synergizing code coverage and gameplay intent: Coverage-aware game playtesting with LLM-guided reinforcement learning